



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

《电子信息技术虚拟仿真综合实践 B》

综合实验报告

基于 Sobel/Prewitt 算子的 FPGA 边缘检测图像处理

指导教师	高宇
学生姓名	陈文轩
学生学号	23040447
学生班级	23186011
学生专业	智能硬件与系统 (电子信息工程)

2025 年 12 月 2 日

摘 要

本实验基于 FPGA 平台实现了图像边缘检测系统，采用 Sobel 和 Prewitt 两种经典边缘检测算子，对存储在 ROM 中的图像进行实时处理并在 LCD 屏幕上显示。系统采用流水线架构，实现了从原始 RGB 图像到边缘检测结果的完整处理链路，包括坐标映射与图像缩放、RGB 转灰度预处理、 3×3 像素窗口构建以及卷积运算等核心功能模块。

实验参考 OpenCV 图像处理库的算法原理，将软件函数硬件化（Soft Core Hardening），在 FPGA 上实现了与 `cv::Sobel()` 和 Prewitt 算子等效的硬件加速处理。通过拨码开关可实时切换不同的边缘检测算法，直观对比各算法的检测效果。本设计还针对梯度幅值计算中绝对值和近似法存在的 45° 方向边缘高估问题，提出了加权近似法的优化方案。

关键词：FPGA；边缘检测；Sobel 算子；Prewitt 算子；图像处理；硬件加速

1 系统方案

1.1 系统原理介绍

边缘检测是图像处理中的基础操作，其核心思想是检测图像中灰度值发生急剧变化的位置。在数学上，边缘对应于图像灰度函数的一阶导数极大值点或二阶导数过零点。

本系统采用基于梯度的边缘检测方法，使用 Sobel 和 Prewitt 两种算子计算图像的水平 and 垂直方向梯度，通过梯度幅值判断边缘强度。系统整体架构如图1所示。

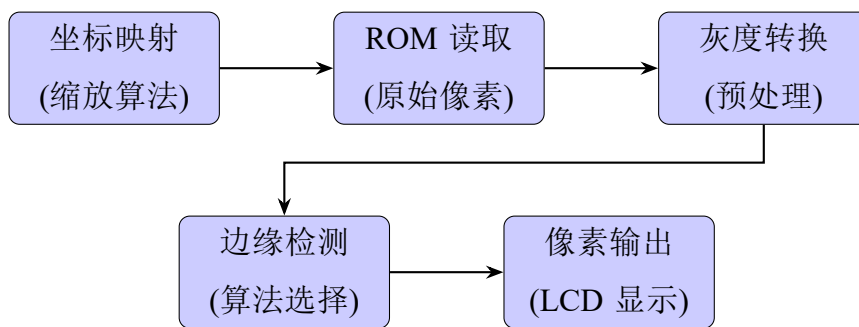


图 1 系统架构框图

1.2 方案论证与比较

1.2.1 边缘检测算子比较

表1对比了 Sobel 和 Prewitt 两种边缘检测算子的特性。

表 1 Sobel 与 Prewitt 算子特性对比

特性	Sobel 算子	Prewitt 算子
中心行权重	2	1
对噪声敏感度	较低	较高
边缘平滑度	更平滑	更锐利
计算复杂度	需要移位操作	仅需加法
适用场景	噪声较多的图像	清晰图像

1.2.2 梯度幅值计算方法比较

精确的梯度幅值计算公式为：

$$|G| = \sqrt{G_x^2 + G_y^2} \quad (1)$$

由于 FPGA 中实现平方根运算消耗大量资源，本设计采用绝对值和近似：

$$|G| \approx |G_x| + |G_y| \quad (2)$$

表2对比了不同梯度幅值计算方法的特性。

表 2 梯度幅值计算方法对比

方法	公式	45° 误差	资源消耗
精确计算	$\sqrt{G_x^2 + G_y^2}$	0%	最高
绝对值和	$ G_x + G_y $	+41%	最低
最大值法	$\max(G_x , G_y)$	-29%	低
加权近似	$\max + \frac{\min}{2}$	$\pm 12\%$	中等

2 各模块详细设计与思路

2.1 坐标映射与图像缩放模块

本模块实现将 LCD 屏幕坐标映射到原始图像坐标，采用最近邻插值算法实现 4 倍放大显示。

参数配置：

- (1) 原始图像尺寸：100×100 像素
- (2) 显示尺寸：400×400 像素（4 倍放大）
- (3) 屏幕分辨率：800×480 像素
- (4) 图像居中位置：(200, 40)

最近邻插值原理：

最近邻插值是最简单的图像缩放算法，其核心思想是：对于目标图像中的每个像素点，直接找到其在源图像中对应的最近像素点，将该像素值作为目标像素值。

在 4 倍放大的场景下，目标图像的坐标范围为 $[0, 399]$ ，需要映射到源图像坐标范围 $[0, 99]$ 。设目标图像坐标为 dst_coord ，源图像坐标为 src_coord ，缩放因子为 $scale_factor = 4$ ，则映射关系为：

$$src_coord = \lfloor \frac{dst_coord}{scale_factor} \rfloor = \lfloor \frac{dst_coord}{4} \rfloor \quad (3)$$

其中 $\lfloor \cdot \rfloor$ 表示向下取整。例如，目标坐标 $dst_coord \in [0, 3]$ 映射到源坐标 $src_coord = 0$ ， $dst_coord \in [4, 7]$ 映射到 $src_coord = 1$ ，以此类推。这样，每个源像素被复制到 $4 \times 4 = 16$ 个目标像素位置。

在硬件实现中，由于 $4 = 2^2$ ，除以 4 的运算可以通过右移 2 位实现，即 $dst_coord \gg 2$ ，无需使用除法器，极大地节省了硬件资源。

```

1 // 计算相对于放大图像的坐标
2 wire [10:0] rel_x_scaled = x_pos - IMG_X; // 0~399
3 wire [10:0] rel_y_scaled = y_pos - IMG_Y; // 0~399
4
5 // 最近邻插值：除以缩放因子4（右移2位）
6 wire [6:0] src_x = rel_x_scaled[8:2]; // 0~99
7 wire [6:0] src_y = rel_y_scaled[8:2]; // 0~99
8
9 // ROM地址计算
10 assign rom_addr = src_y * 100 + src_x; // 0~9999

```

Code Listing 1: 坐标映射模块代码

2.2 RGB 转灰度模块

边缘检测算法通常在灰度图像上进行，因此需要将 RGB 彩色图像转换为灰度图像。

参考 OpenCV 函数：`cv::cvtColor(src, gray, COLOR_RGB2GRAY)`

标准灰度化公式：

$$Gray = 0.299 \times R + 0.587 \times G + 0.114 \times B \quad (4)$$

FPGA 定点优化：

由于 FPGA 中浮点运算需要消耗大量逻辑资源且延迟较高，因此采用定点整数近似方法。将浮点系数乘以 $256 = 2^8$ 并取整，得到整数系数：

$$0.299 \times 256 \approx 77, 0.587 \times 256 \approx 150, 0.114 \times 256 \approx 29$$

定点灰度化公式为：

$$Gray = \frac{77 \times R + 150 \times G + 29 \times B}{256} \quad (5)$$

其中除以 256 可通过右移 8 位实现，即取计算结果的高 8 位。

系数验证： $77/256 \approx 0.301$ ， $150/256 \approx 0.586$ ， $29/256 \approx 0.113$ ，与标准系数误差均小于 1%，满足工程精度要求。

```
1 wire [7:0] r = rom_data[23:16];
2 wire [7:0] g = rom_data[15:8];
3 wire [7:0] b = rom_data[7:0];
4
5 wire [15:0] gray_calc = r * 77 + g * 150 + b * 29;
6 wire [7:0] gray = gray_calc[15:8]; // 右移8位 = 除以256
```

Code Listing 2: RGB 转灰度模块代码

2.3 3×3 像素窗口构建模块

边缘检测卷积运算需要获取当前像素及其 8 邻域像素，形成 3×3 窗口。

窗口结构：

$$\begin{bmatrix} p_{00} & p_{01} & p_{02} \\ p_{10} & p_{11} & p_{12} \\ p_{20} & p_{21} & p_{22} \end{bmatrix} \quad (6)$$

实现原理：

- (1) 使用两个行缓存 line_buf0 和 line_buf1 存储前两行的灰度值
- (2) 当前行使用移位寄存器实时获取
- (3) 每个时钟周期，9 个像素值同时更新，形成完整的 3×3 窗口

```
1 // 行缓存
2 reg [7:0] line_buf0 [0:IMG_WIDTH-1]; // 前2行缓存
3 reg [7:0] line_buf1 [0:IMG_WIDTH-1]; // 前1行缓存
4
```

```

5 // 窗口构建逻辑
6 always @(posedge lcd_clk) begin
7     if (in_image_d1) begin
8         // 从行缓存读取并移位
9         p02 <= p01; p01 <= p00; p00 <= line_buf0[src_x_d1];
10        p12 <= p11; p11 <= p10; p10 <= line_buf1[src_x_d1];
11        p22 <= p21; p21 <= p20; p20 <= gray;
12
13        // 更新行缓存
14        line_buf0[src_x_d1] <= line_buf1[src_x_d1];
15        line_buf1[src_x_d1] <= gray;
16    end
17 end
18
19 // 窗口有效标志
20 wire window_valid = in_image_d3 && (src_x_d3 >= 7'd2) &&
    (src_y_d3 >= 7'd2);

```

Code Listing 3: 3×3 窗口构建代码

2.4 Sobel 边缘检测模块

参考 OpenCV 函数: `cv::Sobel(src, dst, ddepth, dx, dy)`

Sobel 算子定义:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} \quad (7)$$

卷积计算公式:

$$G_x = (p_{02} - p_{00}) + 2 \times (p_{12} - p_{10}) + (p_{22} - p_{20}) \quad (8)$$

$$G_y = (p_{00} + 2 \times p_{01} + p_{02}) - (p_{20} + 2 \times p_{21} + p_{22}) \quad (9)$$

梯度幅值:

$$|G| = |G_x| + |G_y| \quad (10)$$

```

1 // Sobel Gx 计算: 水平方向梯度
2 wire signed [10:0] sobel_gx_row0 = $signed({3'b0, p02}) -
    $signed({3'b0, p00});
3 wire signed [10:0] sobel_gx_row1 = ($signed({3'b0, p12}) -
    $signed({3'b0, p10})) <<< 1;
4 wire signed [10:0] sobel_gx_row2 = $signed({3'b0, p22}) -
    $signed({3'b0, p20});
5 wire signed [11:0] sobel_gx = sobel_gx_row0 + sobel_gx_row1
    + sobel_gx_row2;
6
7 // Sobel Gy 计算: 垂直方向梯度
8 wire signed [10:0] sobel_gy_top = $signed({3'b0, p00}) +
    ($signed({3'b0, p01}) <<< 1) + $signed({3'b0, p02});
9 wire signed [10:0] sobel_gy_bot = $signed({3'b0, p20}) +
    ($signed({3'b0, p21}) <<< 1) + $signed({3'b0, p22});
10 wire signed [11:0] sobel_gy = sobel_gy_top - sobel_gy_bot;
11
12 // 梯度幅值
13 wire [11:0] sobel_gx_abs = (sobel_gx < 0) ? -sobel_gx :
    sobel_gx;
14 wire [11:0] sobel_gy_abs = (sobel_gy < 0) ? -sobel_gy :
    sobel_gy;
15 wire [12:0] sobel_mag_full = sobel_gx_abs + sobel_gy_abs;
16
17 // 归一化到8位
18 wire [7:0] sobel_mag = (sobel_mag_full > 13'd2040) ? 8'd255
    : sobel_mag_full[10:3];
19
20 // 阈值化输出 (白底黑边)
21 wire [23:0] data_sobel = (window_valid && sobel_mag >
    8'd30) ? 24'h000000 : 24'hFFFFFF;

```


Code Listing 4: Sobel 边缘检测代码

2.5 Prewitt 边缘检测模块

Prewitt 算子定义：

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -1 & 0 & +1 \\ -1 & 0 & +1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{bmatrix} \quad (11)$$

卷积计算公式：

$$G_x = (p_{02} - p_{00}) + (p_{12} - p_{10}) + (p_{22} - p_{20}) \quad (12)$$

$$G_y = (p_{00} + p_{01} + p_{02}) - (p_{20} + p_{21} + p_{22}) \quad (13)$$

Prewitt 算子与 Sobel 算子的主要区别在于中心行的权重为 1 而非 2，因此对噪声更敏感，但边缘更锐利。

```

1 // Prewitt Gx 计算：均匀权重
2 wire signed [10:0] prewitt_gx_row0 = $signed({3'b0, p02}) -
    $signed({3'b0, p00});
3 wire signed [10:0] prewitt_gx_row1 = $signed({3'b0, p12}) -
    $signed({3'b0, p10});
4 wire signed [10:0] prewitt_gx_row2 = $signed({3'b0, p22}) -
    $signed({3'b0, p20});
5 wire signed [11:0] prewitt_gx = prewitt_gx_row0 +
    prewitt_gx_row1 + prewitt_gx_row2;
6
7 // Prewitt Gy 计算：均匀权重
8 wire signed [10:0] prewitt_gy_top = $signed({3'b0, p00}) +
    $signed({3'b0, p01}) + $signed({3'b0, p02});
9 wire signed [10:0] prewitt_gy_bot = $signed({3'b0, p20}) +
    $signed({3'b0, p21}) + $signed({3'b0, p22});
10 wire signed [11:0] prewitt_gy = prewitt_gy_top -

```

```

    prewitt_gy_bot;

11
12 // 梯度幅值
13 wire [11:0] prewitt_gx_abs = (prewitt_gx < 0) ? -prewitt_gx
    : prewitt_gx;
14 wire [11:0] prewitt_gy_abs = (prewitt_gy < 0) ? -prewitt_gy
    : prewitt_gy;
15 wire [12:0] prewitt_mag_full = prewitt_gx_abs +
    prewitt_gy_abs;

16
17 // 归一化到8位
18 wire [7:0] prewitt_mag = (prewitt_mag_full > 13'd1530) ?
    8'd255 : prewitt_mag_full[10:3];

19
20 // 阈值化输出（白底黑边）
21 wire [23:0] data_prewitt = (window_valid && prewitt_mag >
    8'd25) ? 24'h000000 : 24'hFFFFFF;

```

Code Listing 5: Prewitt 边缘检测代码

3 电路设计（Verilog HDL 设计）

3.1 整体设计

系统采用模块化设计，主要包含以下功能模块：

- (1) **lcd_pll**: 时钟 PLL 模块，将 50MHz 系统时钟分频为 33MHz LCD 驱动时钟
- (2) **lcd_driver**: LCD 驱动模块，生成 LCD 时序信号
- (3) **lcd_display**: 显示控制模块，包含图像处理和边缘检测功能
- (4) **logo_rom**: 图像 ROM 存储模块

3.2 各模块设计

3.2.1 时钟及分频部分

LCD 驱动时钟由 PLL 模块生成，将系统时钟 50MHz 分频为 33MHz。

```
1 lcd_pll u_lcd_pll (
2     .inclk0 (sys_clk),      // 输入 50MHz
3     .c0      (lcd_clk)      // 输出 33MHz
4 );
```

Code Listing 6: PLL 时钟模块例化

3.2.2 LCD 驱动模块

LCD 驱动模块负责生成 LCD 所需的时序信号，包括行同步 (HSYNC)、场同步 (VSYNC)、数据使能 (DE) 等。

3.2.3 边缘检测处理流水线

系统采用流水线架构，处理延迟为 3 个时钟周期：

表 3 流水线时序

时钟周期	T0	T1	T2	T3
坐标计算	✓			
ROM 读取		✓		
灰度转换			✓	
边缘检测				✓

3.2.4 输出选择模块

通过拨码开关选择不同的显示模式：

表 4 开关功能映射

switches[2:0]	功能	输出数据
3'b001	Sobel 边缘检测	data_sobel
3'b010	Prewitt 边缘检测	data_prewitt
其他	原图显示	rom_data

4 设计拓展：梯度幅值计算优化

4.1 问题分析

绝对值和近似法 $|G| = |G_x| + |G_y|$ 存在一个问题：当水平和垂直方向都有中等强度的边缘时，两者叠加可能导致误判为强边缘。

示例分析：

考虑 $(G_x, G_y) = (70, 70)$ 的情况，即 45° 方向上水平和垂直分量相等的边缘：

精确幅值： $|G| = \sqrt{70^2 + 70^2} = \sqrt{9800} \approx 99$

绝对值和： $|G| \approx 70 + 70 = 140$

可以看出，绝对值和近似法产生了约 $\frac{140-99}{99} \times 100\% \approx 41\%$ 的高估误差。这意味着 45° 方向的边缘会被显著高估，可能导致本不应检测为边缘的区域被误判为强边缘。

4.2 加权近似法解决方案

参考 OpenCV 的优化策略，采用加权近似法：

$$|G| \approx \max(|G_x|, |G_y|) + 0.5 \times \min(|G_x|, |G_y|) \quad (14)$$

或更精确的版本：

$$|G| \approx 0.875 \times \max + 0.5 \times \min \quad (15)$$

硬件实现优化：

由于 $0.5 = 2^{-1}$ ，可以通过右移 1 位实现，无需乘法器：

$$|G| = \max + (\min \gg 1) \quad (16)$$

```

1 // 计算绝对值
2 wire [10:0] gx_abs = (gx < 0) ? (-gx) : gx[10:0];
3 wire [10:0] gy_abs = (gy < 0) ? (-gy) : gy[10:0];
4
5 // 计算max和min
6 wire [10:0] g_max = (gx_abs > gy_abs) ? gx_abs : gy_abs;
7 wire [10:0] g_min = (gx_abs > gy_abs) ? gy_abs : gx_abs;
8
9 // 加权近似: G = max + min/2 (误差控制在±12%以内)
10 wire [11:0] gradient = g_max + {1'b0, g_min[10:1]};
11
12 // 更精确版本 (误差<6%):
13 // G = max + min/4 + min/8 = max + 3*min/8
14 // wire [11:0] gradient = g_max + {2'b0, g_min[10:2]} +
    {3'b0, g_min[10:3]};

```

Code Listing 7: 加权近似法实现代码

5 测试结果

5.1 原图显示功能

当所有开关为 0 时，系统显示 4 倍放大的原始彩色图像，图像居中显示在 800×480 的 LCD 屏幕上。图2展示了存储在 ROM 中的原始图像。

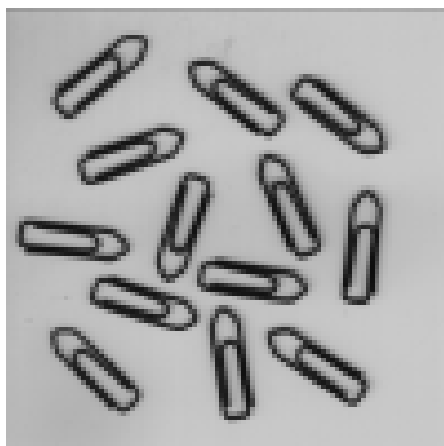


图 2 原始图像（100×100 像素）

图3展示了图像在 FPGA 开发板 LCD 屏幕上的实际显示效果，可以看到图像被 4 倍放大后居中显示在白色背景上。

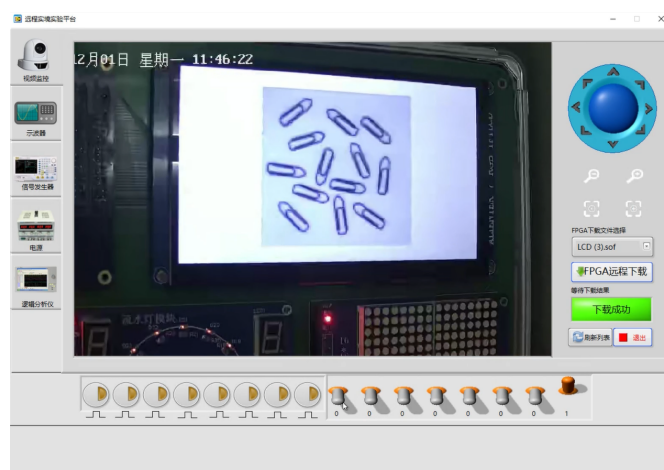


图 3 图像在 FPGA LCD 屏幕上的显示效果

5.2 Sobel 边缘检测功能

拨动 Switch[0] 后，系统使用 Sobel 算子进行边缘检测，检测结果以白底黑边的形式显示。Sobel 算子对噪声有一定的抑制能力，检测出的边缘较为平滑。图4展示了 Sobel 边缘检测的效果。



图 4 Sobel 边缘检测效果

5.3 Prewitt 边缘检测功能

拨动 Switch[1] 后, 系统使用 Prewitt 算子进行边缘检测。与 Sobel 相比, Prewitt 检测出的边缘更为锐利, 但对噪声更敏感。图5展示了 Prewitt 边缘检测的效果。



图 5 Prewitt 边缘检测效果

5.4 功能切换测试

通过实时切换拨码开关, 可以即时观察不同边缘检测算法的效果差异, 系统响应迅速, 无明显延迟。对比图4和图5可以发现, Sobel 算子检测的边缘更加平滑连续, 而 Prewitt 算子检测的边缘更加锐利清晰。

6 结论

本实验成功实现了基于 FPGA 的图像边缘检测系统, 主要完成以下工作:

(1) 实现了完整的图像处理流水线, 包括坐标映射、图像缩放、RGB 转灰度、 3×3 窗口构建等模块

(2) 采用 Sobel 和 Prewitt 两种经典边缘检测算子, 实现了与 OpenCV 软件库等效的硬件加速处理

(3) 通过拨码开关实现算法的实时切换, 便于对比不同算法的效果

(4) 针对梯度幅值计算中绝对值和近似法的 45° 方向高估问题, 提出了加权近似法的优化方案

本设计将软件图像处理算法硬件化, 充分发挥了 FPGA 并行处理的优势, 实现了实时图像边缘检测, 对理解数字图像处理原理和 FPGA 设计方法具有重要的实践意义。

7 参考文献

- [1] 冈萨雷斯. 数字图像处理 (第三版) [M]. 电子工业出版社, 2011.
- [2] OpenCV 官方文档. Image Filtering—Sobel Derivatives[EB/OL]. <https://docs.opencv.org/>
- [3] 夏宇闻. Verilog 数字系统设计教程 (第四版) [M]. 北京航空航天大学出版社, 2017.
- [4] Intel Corporation. Cyclone IV Device Handbook[EB/OL]. <https://www.intel.com/>

附录：lcd_display.v 屏幕应用层程序

```
1 module lcd_display(  
2     input          lcd_clk,          // LCD驱动时钟  
3     input          sys_rst_n,        // 复位信号  
4     input [3:0]    switches,         // B3,B2,B1,B0 开关输入  
5     input [10:0]   pixel_xpos,       // 像素点横坐标(0-799)  
6     input [10:0]   pixel_ypos,       // 像素点纵坐标(0-479)  
7     output reg [23:0] pixel_data // 像素点数据(RGB888)  
8 );  
9  
10 //--  
11 // Parameter definition  
12 //---  
13 localparam H_DISP = 11'd800; // 水平分辨率  
14 localparam V_DISP = 11'd480; // 垂直分辨率  
15  
16 // Logo 图片参数 (从LOGO.mif读取)  
17 localparam IMG_WIDTH  = 11'd100; // 原始图片宽度  
18 localparam IMG_HEIGHT = 11'd100; // 原始图片高度  
19  
20 // 缩放参数 (4倍放大: 100x100 -> 400x400)  
21 localparam SCALE_FACTOR = 3'd4; // 缩放倍数  
22 localparam SCALED_WIDTH  = 11'd400; // 缩放后宽度  
23 localparam SCALED_HEIGHT = 11'd400; // 缩放后高度  
24  
25 // 居中显示位置参数 (屏幕800x480, 图像400x400)  
26 localparam IMG_X = 11'd200; // 图像左上角X坐标 (居中)  
27 localparam IMG_Y = 11'd40; // 图像左上角Y坐标 (居中)  
28  
29 // 颜色定义  
30 localparam COLOR_BG = 24'hFFFFFF; // 白色背景  
31  
32 // ROM相关信号
```

```
33 wire [13:0] rom_addr;
34 wire [23:0] rom_data;
35
36 wire [10:0] x_pos = (pixel_xpos == 11'd0) ? 11'd0 :
    pixel_xpos - 11'd1;
37 wire [10:0] y_pos = (pixel_ypos == 11'd0) ? 11'd0 :
    pixel_ypos - 11'd1;
38
39 wire in_active_area = (pixel_xpos >= 11'd1) && (pixel_xpos
    <= H_DISP) &&
40
    (pixel_ypos >= 11'd1) && (pixel_ypos
    <= V_DISP);
41
42 // 判断当前像素是否在缩放后的图片范围内 (400x400)
43 wire [10:0] rel_x_scaled = x_pos - IMG_X;
44 wire [10:0] rel_y_scaled = y_pos - IMG_Y;
45 wire in_image = (x_pos >= IMG_X) && (x_pos < IMG_X +
    SCALED_WIDTH) &&
46
    (y_pos >= IMG_Y) && (y_pos < IMG_Y +
    SCALED_HEIGHT);
47
48 // 缩放算法：最近邻插值
49 wire [6:0] src_x = rel_x_scaled[8:2]; // 除以4
50 wire [6:0] src_y = rel_y_scaled[8:2]; // 除以4
51
52 assign rom_addr = (src_y * 7'd100) + src_x;
53
54 //----
55 // Logo ROM实例化
56 //-----
57 logo_rom u_logo_rom (
58     .address (rom_addr),
59     .clock    (lcd_clk),
60     .q        (rom_data)
```

```
61 );  
62  
63 // 寄存in_image信号以匹配ROM延迟  
64 reg in_image_d1;  
65 always @(posedge lcd_clk) begin  
66     in_image_d1 <= in_image;  
67 end  
68  
69 //-----  
70 // 图像处理流水线 - 边缘检测算法  
71 //-----  
72 wire [7:0] r = rom_data[23:16];  
73 wire [7:0] g = rom_data[15:8];  
74 wire [7:0] b = rom_data[7:0];  
75  
76 // 灰度化  
77 wire [15:0] gray_calc = r * 77 + g * 150 + b * 29;  
78 wire [7:0] gray = gray_calc[15:8];  
79  
80 //-----  
81 // 3x3 像素窗口缓存  
82 //-----  
83 reg [7:0] line_buf0 [0:IMG_WIDTH-1];  
84 reg [7:0] line_buf1 [0:IMG_WIDTH-1];  
85  
86 reg [7:0] p00, p01, p02;  
87 reg [7:0] p10, p11, p12;  
88 reg [7:0] p20, p21, p22;  
89  
90 reg in_image_d2, in_image_d3;  
91 reg [6:0] src_x_d1, src_x_d2, src_x_d3;  
92 reg [6:0] src_y_d1, src_y_d2, src_y_d3;  
93  
94 always @(posedge lcd_clk) begin
```

```
95     in_image_d2 <= in_image_d1;
96     in_image_d3 <= in_image_d2;
97     src_x_d1 <= src_x;
98     src_x_d2 <= src_x_d1;
99     src_x_d3 <= src_x_d2;
100    src_y_d1 <= src_y;
101    src_y_d2 <= src_y_d1;
102    src_y_d3 <= src_y_d2;
103 end
104
105 // 窗口构建
106 always @(posedge lcd_clk) begin
107     if (in_image_d1) begin
108         p02 <= p01; p01 <= p00; p00 <= line_buf0[src_x_d1];
109         p12 <= p11; p11 <= p10; p10 <= line_buf1[src_x_d1];
110         p22 <= p21; p21 <= p20; p20 <= gray;
111         line_buf0[src_x_d1] <= line_buf1[src_x_d1];
112         line_buf1[src_x_d1] <= gray;
113     end
114 end
115
116 wire window_valid = in_image_d3 && (src_x_d3 >= 7'd2) &&
    (src_y_d3 >= 7'd2);
117
118 //-----
119 // Sobel 边缘检测
120 //-----
121 wire signed [10:0] sobel_gx_row0 = $signed({3'b0, p02}) -
    $signed({3'b0, p00});
122 wire signed [10:0] sobel_gx_row1 = ($signed({3'b0, p12}) -
    $signed({3'b0, p10})) <<< 1;
123 wire signed [10:0] sobel_gx_row2 = $signed({3'b0, p22}) -
    $signed({3'b0, p20});
124 wire signed [11:0] sobel_gx = sobel_gx_row0 + sobel_gx_row1
```

```

    + sobel_gx_row2;

125
126 wire signed [10:0] sobel_gy_top = $signed({3'b0, p00}) +
    ($signed({3'b0, p01}) <<< 1) + $signed({3'b0, p02});
127 wire signed [10:0] sobel_gy_bot = $signed({3'b0, p20}) +
    ($signed({3'b0, p21}) <<< 1) + $signed({3'b0, p22});
128 wire signed [11:0] sobel_gy = sobel_gy_top - sobel_gy_bot;
129
130 wire [11:0] sobel_gx_abs = (sobel_gx < 0) ? -sobel_gx :
    sobel_gx;
131 wire [11:0] sobel_gy_abs = (sobel_gy < 0) ? -sobel_gy :
    sobel_gy;
132 wire [12:0] sobel_mag_full = sobel_gx_abs + sobel_gy_abs;
133 wire [7:0] sobel_mag = (sobel_mag_full > 13'd2040) ? 8'd255
    : sobel_mag_full[10:3];
134 wire [23:0] data_sobel = (window_valid && sobel_mag >
    8'd30) ? 24'h000000 : 24'hFFFFFF;
135
136 //-----
137 // Prewitt 边缘检测
138 //-----
139 wire signed [10:0] prewitt_gx_row0 = $signed({3'b0, p02}) -
    $signed({3'b0, p00});
140 wire signed [10:0] prewitt_gx_row1 = $signed({3'b0, p12}) -
    $signed({3'b0, p10});
141 wire signed [10:0] prewitt_gx_row2 = $signed({3'b0, p22}) -
    $signed({3'b0, p20});
142 wire signed [11:0] prewitt_gx = prewitt_gx_row0 +
    prewitt_gx_row1 + prewitt_gx_row2;
143
144 wire signed [10:0] prewitt_gy_top = $signed({3'b0, p00}) +
    $signed({3'b0, p01}) + $signed({3'b0, p02});
145 wire signed [10:0] prewitt_gy_bot = $signed({3'b0, p20}) +
    $signed({3'b0, p21}) + $signed({3'b0, p22});

```

```
146 wire signed [11:0] prewitt_gy = prewitt_gy_top -
    prewitt_gy_bot;
147
148 wire [11:0] prewitt_gx_abs = (prewitt_gx < 0) ? -prewitt_gx
    : prewitt_gx;
149 wire [11:0] prewitt_gy_abs = (prewitt_gy < 0) ? -prewitt_gy
    : prewitt_gy;
150 wire [12:0] prewitt_mag_full = prewitt_gx_abs +
    prewitt_gy_abs;
151 wire [7:0] prewitt_mag = (prewitt_mag_full > 13'd1530) ?
    8'd255 : prewitt_mag_full[10:3];
152 wire [23:0] data_prewitt = (window_valid && prewitt_mag >
    8'd25) ? 24'h000000 : 24'hFFFFFF;
153
154 //-----
155 // 像素显示逻辑
156 //-----
157 always @(posedge lcd_clk or negedge sys_rst_n) begin
158     if (!sys_rst_n) begin
159         pixel_data <= COLOR_BG;
160     end else begin
161         if (!in_active_area) begin
162             pixel_data <= COLOR_BG;
163         end else begin
164             if (in_image_d1) begin
165                 case (switches[1:0])
166                     2'b01: pixel_data <= data_sobel;    //
167                         Switch[0]: Sobel
168                     2'b10: pixel_data <= data_prewitt;  //
169                         Switch[1]: Prewitt
170                     default: pixel_data <= rom_data;    // Default:
171                         原图
172                 endcase
173             end else begin
```

```
171         pixel_data <= COLOR_BG;  
172     end  
173 end  
174 end  
175 end  
176  
177 endmodule
```

Code Listing 8: lcd_display.v 完整代码